HiveOps Machine Learning Architecture

This document outlines the specialized Machine Learning (ML) architecture powering the HiveOps geospatial decision-support system. HiveOps employs an "offline-first" predictive pipeline, ensuring high-performance visualization without real-time compute overhead.

1. ML Objective

The objective of the ML engine is to **predict the environmental suitability score for flowering** across a geographic region using high-resolution spatial and climate features. This score enables beekeepers to identify optimal hive placement locations based on projected floral resources rather than static historical observations.

2. Input Data (Features)

The model utilizes a multi-dimensional feature set to understand the biological requirements of local flora:

Feature	Justification
Temperature	Determines metabolic rates of plants and nectar secretion windows.
Humidity	Influences nectar concentration and pollen viability.
Rainfall	Indicates soil moisture availability for long-term plant growth.
Latitude/Longitude	Captures geographic variance, soil types, and regional biomes.
Month	Accounts for seasonality and phenological cycles of flowering species.

3. Target Variable

The model targets a continuous **NDVI-based (Normalized Difference Vegetation Index) score**.

- **Nature:** Continuous regression (not classification).
- **Transformation:** Raw NDVI values are normalized and converted into a **0–100 Suitability Score**, where 0 represents a barren environment and 100 represents peak flowering density.

- **Logic:** No arbitrary thresholds are applied; the model provides a granular gradient of suitability across the entire spectrum.

4. Model Design

- **Model Type:** Random Forest Regressor.
- **Rationale:** Regression is required to produce the continuous 0–100 scores needed for smooth heatmap gradients.
- **Suitability:** Random Forest effectively captures the complex, non-linear relationships between climate variables (e.g., the specific interaction between high rainfall and specific temperatures) without the risk of overfitting common in deeper architectures when dealing with regional tabular data.

5. Training Process

1. **Dataset Generation:** Grid-based sampling is performed across the target region, pairing historical climate data with observed satellite NDVI indices.
2. **Data Cleaning:** Removal of outliers caused by cloud cover or sensor noise in satellite imagery.
3. **Split:** 80/20 Train-Test split.
4. **Evaluation:** Performance is measured using **R^2** (variance explained) and **MAE** (Mean Absolute Error) to ensure the 0-100 score remains accurate within a ±5% margin.

6. Windy-Style Data Pipeline (Critical)

HiveOps operates on an offline precomputation pipeline. ML execution is decoupled from user interaction to ensure sub-second page loads.

1. **ML (Offline):** The Random Forest model runs on a high-compute server, generating predictions for every coordinate in a high-resolution grid.
2. **Grid-to-Tile:** Predicted scores are grouped into a standardized tile system (z/x/y).
3. **Database Storage:** These precomputed tiles are stored as document structures in MongoDB.
4. **API Serving:** When a user pans the map, the API fetches the relevant precomputed tiles.
5. **Zero-Latency:** No ML inference occurs when the user clicks or moves the map.

7. Tile Generation

- **Structure:** Each geographic tile is converted into a **32x32 matrix** of suitability scores.

- **Interpolation:** Bi-linear interpolation is used during the generation phase to smooth transitions between grid points, preventing a "blocky" appearance.
- **Storage:** Data is stored as compressed arrays in MongoDB for rapid retrieval.

8. Frontend Connection

- **Data Fetch:** The React frontend identifies the visible bounding box and calls `/api/tile`.
- **Rendering:** The frontend receives the 32x32 grid and passes it to a **WebGL-powered BitmapLayer**.
- **Execution:** The GPU handles the color mapping (e.g., 0=Red, 100=Green).
- **Constraint:** No ML logic or heavy mathematical computation exists in the client-side code; the browser is strictly a visualization layer.

9. Why This Architecture?

- **Performance:** Eliminating real-time inference removes "lag" usually associated with ML-heavy apps.
- **Scalability:** Serving precomputed JSON/Tiles scales infinitely through CDNs, unlike live Python-based ML environments.
- **User Experience:** Provides a fluid, "Windy-style" interactive experience where heatmap data appears instantly as the user scrolls.

Summary

ML powers the HiveOps system by generating precomputed environmental intelligence that is visualized as a heatmap for decision making. By moving the Random Forest Regressor offline